uc3m | Universidad Carlos III de Madrid

Máster Universitario en Robótica y Automatización

Robots de campo | Curso 2025-2026

Trabajo Práctico

NAVEGACIÓN PUNTO A PUNTO EN TERRENO DESNIVELADO UTILIZANDO EL SIMULADOR COPPELIASIM

Grupo:

Mario Gómez Sánchez-Celemín

03 de Marzo del 2026

1. Algoritmos utilizados

1.1. PID

Desde su aparición a principios del siglo XX, el controlador PID se ha convertido en el regulador por excelencia, abarcando más del cincuenta por ciento de los sistemas de control utilizados en la industria. Este algoritmo emplea la señal de error, que es la diferencia entre la señal de referencia y la señal de salida, para determinar la acción de control que se debe aplicar a la planta en cada instante de tiempo. El sistema de control PID combina las acciones Proporcional, Integral y Derivativa para lograr el comportamiento deseado. La ecuación ideal del PID se presenta en la ecuación 1.1.

$$u(t) = K\left[e(t) + \frac{1}{T_i}\int_0^t e(\tau)\,d\tau + T_d\frac{de(t)}{dt}\right] \tag{1.1}$$

Donde $u(t)$ es la señal de control, $e(t)$ la señal de error, K es la ganancia proporcional, T_i el tiempo integral y T_d el tiempo derivativo. Esta ecuación suele reescribirse para facilitar los cálculos:

$$u(t) = K_p e(t) + K_i\int_0^t e(t)\,dt + K_d\left(\frac{de(t)}{dt}\right) \tag{1.2}$$

Alternativamente, la función de transferencia del controlador PID en el dominio de Laplace se expresa como:

$$C_PID(s) = K\left(\frac{1 + sT_i + s^2T_iT_d}{sT_i}\right) = K_p + \frac{K_i}{s} + K_d s \tag{1.3}$$

Cada una de las acciones de este regulador tiene un efecto específico en el Lugar de las Raíces (LDR) del sistema controlado. En términos generales, el PID se basa en añadir polos y ceros en ubicaciones específicas para modificar la respuesta del sistema.

- **Acción proporcional**: la acción proporcional, descrita en la ecuación 1.4, se utiliza cuando la acción de control debe ser proporcional a la señal de error. A continuación se muestra la acción de control del regulador P.

$$\begin{aligned} \text{Dominio temporal} \qquad & u(t) = K_p e(t) \\ \text{Dominio de Laplace} \qquad & U(s) = K_p E(s) \end{aligned} \tag{1.4}$$

Generalmente, al aumentar el valor de K_p, el sistema se vuelve más rápido y más oscilatorio, y el error disminuye. Sin embargo, un aumento excesivo de este parámetro puede provocar oscilaciones pronunciadas y ralentizar el sistema. Por lo tanto, es necesario encontrar un equilibrio entre la respuesta dinámica y la precisión. Es importante tener en cuenta que, a diferencia de otros reguladores, el LDR no se modifica con este compensador.

- **Acción integral**: la acción integral añade un polo en el origen de la cadena abierta, aumentando en una unidad el orden del sistema y reduciendo en un orden el error en estado estacionario. Sin embargo, este aumento en el orden del sistema también reduce su estabilidad, haciendo la respuesta más lenta, donde la salida ya no es dominada por los polos complejos conjugados. Esta acción siempre va acompañada de la acción proporcional (PI), como se muestra en la ecuación 1.5, o en conjunto con las acciones proporcional e integral (PID).

$$\begin{aligned} &\text{Dominio temporal} && u_c(t) = K_p e(t) + K_i \int_0^t e(t)\,dt \\ &\text{Dominio de Laplace} && U_c(s) = \left[K\left(\frac{1+sT_i}{sT_i}\right)\right] E(s) = \left[K_i\left(\frac{1}{s}\right) + K_p\right] E(s) \end{aligned} \tag{1.5}$$

- **Acción derivativa**: una acción es derivativa cuando la salida es proporcional a la derivada del error. En el contexto del control, la acción derivativa se relaciona con la velocidad de cambio de la variable dependiente. Esto significa que el controlador derivativo responde a la rapidez con la que cambia el error, permitiendo realizar una corrección significativa antes de que el error crezca demasiado. Esta acción es anticipativa, ya que el controlador se adelanta a la tendencia del error. Sin embargo, el control derivativo por sí solo no puede corregir errores en estado estable, por lo que debe utilizarse en conjunto con las acciones proporcional (PD) o proporcional e integral (PID).

  Al analizar el regulador PD de la ecuación 1.6 desde la perspectiva del dominio complejo y del LDR, se observa que introduce un cero en la función de transferencia en lazo abierto, alejando los polos dominantes del eje imaginario, lo que hace que el sistema sea más rápido y estable.

$$\begin{aligned} &\text{Dominio temporal} && u_c(t) = K_p e(t) + K_d \frac{de(t)}{dt} \\ &\text{Dominio de Laplace} && U_c(s) = [K(1+sT_d)]\,E(s) = [K_p + sK_d]\,E(s) \end{aligned} \tag{1.6}$$

Dado que el proceso de diseño del PID no es el objetivo principal de este proyecto, no se abordan en detalle las técnicas de ajuste de este tipo de reguladores, como el uso del LDR, la respuesta en frecuencia o el método de Ziegler-Nichols.

1.2. Pure Pursuit

El algoritmo *Pure Pursuit* es una técnica de seguimiento de trayectorias ampliamente utilizada en robótica móvil y vehículos autónomos. Su objetivo es generar una trayectoria suave que permita al robot seguir un camino previamente definido mediante el cálculo de una curvatura adecuada en cada instante.

El principio fundamental del algoritmo consiste en seleccionar un punto objetivo sobre la trayectoria a una cierta distancia por delante del robot, denominada *lookahead distance*. El robot calcula entonces el arco de circunferencia que conecta su posición actual con dicho punto objetivo y ajusta su curvatura para seguir ese arco.

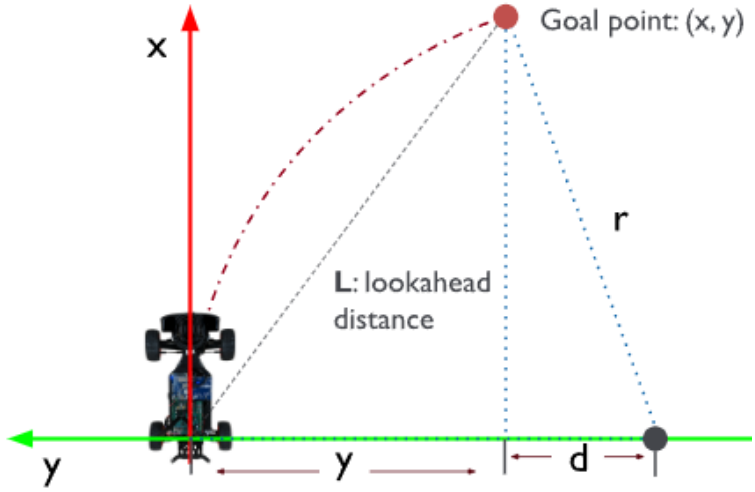


Figura 1.1: Representación geométrica del algoritmo Pure Pursuit

Desde un punto de vista geométrico, el método se basa en la relación entre la posición relativa del punto objetivo y el radio de la circunferencia que describe el movimiento del robot. Considerando la geometría mostrada en la Figura 1.1, se obtiene la siguiente relación:

$$x^2 + d^2 = r^2 \tag{1.7}$$

$$r = |y| + d \tag{1.8}$$

donde r representa el radio del arco de circunferencia que conecta el robot con el punto objetivo, x e y corresponden a las coordenadas relativas del punto objetivo en el sistema de referencia del robot, y d es una distancia auxiliar derivada de la construcción geométrica.

A partir de estas relaciones se obtiene una expresión para el radio del arco:

$$r = \frac{L^2}{2|y|} \tag{1.9}$$

donde L corresponde a la distancia entre el robot y el punto objetivo, también conocida como *lookahead distance*.

Dado que la curvatura γ de una trayectoria circular se define como el inverso del radio, se obtiene:

$$\gamma = \frac{1}{r} = \frac{2|y|}{L^2} \tag{1.10}$$

Esta curvatura se utiliza posteriormente para calcular el comando de dirección o velocidad angular del robot, permitiendo así ajustar su trayectoria hacia el punto objetivo seleccionado.

Cabe destacar que la solución generada por el algoritmo no es única, ya que depende directamente del valor elegido para el parámetro *lookahead distance*. Este parámetro determina la distancia a la cual se selecciona el punto objetivo sobre la trayectoria y tiene un impacto significativo en el comportamiento del sistema. Valores pequeños permiten un seguimiento más preciso de la trayectoria, pero pueden generar movimientos más bruscos o inestables, mientras que valores mayores producen trayectorias más suaves a costa de reducir la precisión del seguimiento.

1.3. Algoritmo de Dijkstra

El algoritmo de Dijkstra es un método clásico de búsqueda de caminos mínimos en grafos ponderados. Fue propuesto por el científico informático neerlandés en 1956 y permite calcular la distancia mínima desde un nodo origen hasta el resto de nodos de un grafo cuyos pesos son no negativos.

En el contexto de robótica móvil y planificación de trayectorias, este algoritmo se utiliza para determinar la ruta óptima entre dos posiciones dentro de un entorno discretizado, normalmente representado mediante un grafo o una rejilla (*grid*). Cada nodo representa una posible posición del robot y las aristas representan las transiciones entre estados vecinos, asociadas a un coste de desplazamiento.

Sea $G = (V, E)$ un grafo compuesto por un conjunto de vértices V y un conjunto de aristas E. A cada arista $(u, v) \in E$ se le asocia un coste $w(u, v)$ que representa el coste de desplazarse desde el nodo u hasta el nodo v. El objetivo del algoritmo es encontrar el camino de coste mínimo desde un nodo inicial s hasta cualquier otro nodo del grafo.

El algoritmo mantiene un conjunto de distancias mínimas estimadas desde el nodo inicial hacia cada nodo del grafo. Inicialmente, la distancia al nodo inicial se define como cero, mientras que el resto de nodos se inicializan con un valor infinito:

$$d(s) = 0, \qquad d(v) = \infty \quad \forall v \neq s \tag{1.11}$$

Durante cada iteración, el algoritmo selecciona el nodo no visitado con menor distancia estimada y actualiza las distancias de sus nodos vecinos mediante la operación conocida como *relajación*:

$$d(v) = \text{mín}\,(d(v), d(u) + w(u, v)) \tag{1.12}$$

donde u es el nodo actualmente seleccionado y v es uno de sus vecinos. Este proceso se repite hasta que todos los nodos han sido visitados o hasta que se alcanza el nodo objetivo.

Una de las características principales del algoritmo de Dijkstra es que garantiza encontrar el camino de coste mínimo siempre que todos los pesos de las aristas sean no negativos. Además, su implementación suele apoyarse en estructuras de datos como colas de prioridad para mejorar la eficiencia computacional.

En aplicaciones de planificación de movimiento, el algoritmo permite generar una trayectoria global óptima sobre el mapa del entorno. Sin embargo, al tratarse de un algoritmo de planificación global, no tiene en cuenta directamente las limitaciones dinámicas del robot ni la presencia de obstáculos dinámicos.

1.4. Algoritmo A*

El algoritmo A* es otro método de búsqueda heurística ampliamente utilizado en problemas de planificación de caminos y navegación autónoma. Fue desarrollado en 1968 y constituye una extensión del algoritmo de Dijkstra que introduce una función heurística para guiar el proceso de búsqueda hacia el objetivo.

Al igual que en el algoritmo de Dijkstra, el entorno se modela mediante un grafo $G = (V, E)$, donde V representa el conjunto de nodos y E el conjunto de aristas que conectan dichos nodos. Cada arista (u, v) tiene asociado un coste $w(u, v)$ que representa el coste de desplazamiento entre los nodos u y v.

La principal diferencia respecto a Dijkstra es que el algoritmo A* utiliza una función de evaluación que combina el coste acumulado desde el nodo inicial hasta el nodo actual con una estimación heurística del coste restante hasta el nodo objetivo. Esta función se define como:

$$f(n) = g(n) + h(n) \tag{1.13}$$

Donde:

- $g(n)$ representa el coste acumulado desde el nodo inicial hasta el nodo actual n.
- $h(n)$ es una función heurística que estima el coste desde el nodo n hasta el nodo objetivo.
- $f(n)$ es la estimación del coste total del camino que pasa por el nodo n.

Durante el proceso de búsqueda, el algoritmo selecciona en cada iteración el nodo con menor valor de $f(n)$ dentro del conjunto de nodos abiertos (*open set*). A continuación, se actualizan los costes de los nodos vecinos mediante un proceso de relajación similar al utilizado en el algoritmo de Dijkstra:

$$g(v) = \text{mín}\,(g(v), g(u) + w(u, v)) \tag{1.14}$$

Donde u es el nodo actualmente expandido y v es uno de sus vecinos.

La eficiencia del algoritmo depende en gran medida de la heurística empleada. En entornos bidimensionales discretizados, es habitual utilizar la distancia euclídea o la distancia Manhattan

como función heurística, ya que proporcionan una estimación razonable del coste restante hasta el objetivo.

Cuando la heurística es admisible, es decir, nunca sobreestima el coste real hasta el objetivo, el algoritmo A* garantiza encontrar el camino óptimo. Gracias a esta propiedad y a su mayor eficiencia respecto al algoritmo de Dijkstra, A* se ha convertido en uno de los métodos más utilizados para planificación de trayectorias en sistemas de navegación autónoma.

2. Desarrollo

En este capítulo se describe la implementación desarrollada para la simulación y evaluación de los algoritmos presentados anteriormente. Para la realización del proyecto se ha utilizado el simulador CoppeliaSim en su versión V4.10.0 rev0 - Edu. La simulación se ha ejecutado sobre un equipo Lenovo Legion 5 con las siguientes características:

- Memoria RAM: 32 GB
- Procesador: AMD Ryzen 9
- Tarjeta gráfica: NVIDIA GeForce RTX 4070
- Sistema operativo: Windows

El control de versiones del proyecto se ha gestionado mediante el sistema Git, lo que permite mantener un historial de cambios y facilitar el desarrollo colaborativo y la reproducibilidad del trabajo. El repositorio completo del proyecto se encuentra disponible en la plataforma de desarrollo *Github* en el siguiente enlace:

`https://github.com/mariogsc12/FieldRobots_RoboticMaster`

2.1. Estructura del repositorio

El proyecto se encuentra organizado en diferentes directorios que contienen los distintos componentes necesarios para la simulación, ejecución y almacenamiento de resultados. Esta estructura permite separar claramente los archivos de simulación, el código fuente, la documentación y los datos generados durante las ejecuciones.

En particular, la carpeta `worlds` contiene los mundos de simulación utilizados en CoppeliaSim, mientras que `documentation` almacena la documentación asociada al proyecto. Por su parte, el directorio `simulation_data` recoge los datos generados durante las simulaciones, tales como trayectorias ejecutadas y planificadas del entorno.

El núcleo del proyecto se encuentra en el directorio `coppeliaSim`, donde se organizan los scripts y módulos necesarios para la comunicación con el simulador y la implementación de los algoritmos de control y planificación.

Dentro del directorio principal `coppeliaSim` se organiza el código de la simulación en distintos subdirectorios según su funcionalidad como se muestra en la Figura 2.1.. La carpeta `control`

contiene el archivo principal encargado de ejecutar la simulación y coordinar los distintos módulos. Por su parte, `starters` incluye scripts en Lua destinados a inicializar la comunicación con la API de CoppeliaSim y ejecutar funcionalidades específicas de la simulación. La carpeta `test` agrupa scripts orientados a la comprobación y validación del correcto funcionamiento de la comunicación con el simulador y de los distintos módulos del sistema. Finalmente, `utils` alberga funciones y módulos auxiliares que simplifican operaciones comunes, matemáticas o de gestión de datos, contribuyendo a una estructura de código más clara y modular.

```
coppeliaSim/
├─ control/
│  ├─ main.py
│  └─ __init__.py
├─ starters/
│  ├─ api_connection_script.lua
│  ├─ check_objects_script.lua
│  └─ store_csv_script.lua
├─ test/
│  ├─ test_connection.py
│  └─ __init__.py
├─ utils/
│  ├─ math.py
│  ├─ utils.py
│  └─ __init__.py
├─ config.yaml
└─ __init__.py
```

Figura 2.1: Estructura de la carpeta coppeliaSim

2.2. Entorno de simulación seleccionado

Se ha utilizado como base el *heightmap* de ejemplo proporcionado por el aula global, al cual se ha incorporado el robot *Robotnik Summit XL* en su versión equipada con ruedas de goma, más robustas y adecuadas para entornos exteriores. Esta configuración permite evaluar el comportamiento del robot en escenarios de simulación representativos de aplicaciones reales.

Para facilitar la observación y análisis de la trayectoria seguida por el robot, se han configurado dos cámaras dentro del entorno simulado: una cámara fija que proporciona una vista general del entorno y permite visualizar la planificación global, y una cámara flotante que sigue al robot desde una cierta elevación, permitiendo observar de manera detallada el seguimiento de la trayectoria y la dinámica del vehículo durante la simulación.

Las características mecánicas, de control y especificaciones técnicas del robot se resumen en la Tabla 2.1.

Mecánica / Control	**Descripción**
Controlador	Arquitectura abierta en ROS
PC integrado	Linux
Comunicación	WiFi 802.11n
Conectividad interna	USB, RS232, GPIO
Conectividad externa	USB, RJ45, 12 VDC, batería
Especificaciones técnicas	**Valores**
Fuente	`www.robotnik.eu`
Referencia	CF.03.23
Dimensiones (Ruedas de goma)	720 x 614 x 416 mm
Peso	65 kg
Capacidad de carga	50 kg
Velocidad máxima	3 m/s
Entorno	Interior / Exterior (IP53 opcional)
Autonomía	Hasta 10 h
Batería	LiFePO4 15Ah @ 48 V
Sistema de tracción	Skid steering / Omni direccional (mecanum)
Motores de tracción	4 x 500 W servomotores sin escobillas
Rango de temperatura	-10°C a +45°C
Máxima pendiente	80 %

Tabla 2.1: Especificaciones técnicas del robot *Robotnik Summit XL*

2.2.1. Cinemática diferencial del robot

Para el control del movimiento del robot se ha empleado un modelo de *cinemática diferencial*, que permite calcular las velocidades individuales de cada rueda a partir de la velocidad lineal v y angular ω del robot.

Aunque el robot dispone de cuatro ruedas, se ha simplificado el análisis considerando dos pares de ruedas (izquierda y derecha), tratándolas como si fueran dos ruedas diferenciales, lo que

reduce la complejidad del problema sin afectar la precisión de la simulación.

La relación entre la velocidad lineal del centro del robot y las velocidades angulares de las ruedas se expresa mediante las siguientes ecuaciones:

$$v = \frac{v_r + v_l}{2} \tag{2.1}$$

$$\omega = \frac{v_r - v_l}{L} \tag{2.2}$$

Donde:

- v es la velocidad lineal del robot.
- ω es la velocidad angular del robot alrededor de su eje vertical.
- v_r y v_l son las velocidades lineales de las ruedas derecha e izquierda, respectivamente.
- L es la distancia entre los dos ejes de ruedas (ancho del robot).

A partir de estas ecuaciones, es posible determinar las velocidades de cada rueda necesarias para que el robot siga la trayectoria deseada en el entorno de simulación.

Cálculo de las coordenadas de inicio y fin

Una vez definido el entorno de simulación, es necesario establecer las coordenadas de inicio y final que se utilizarán dentro de dicho entorno.

Para la determinación de estas coordenadas se emplearán datos derivados del nombre y apellidos, siguiendo un procedimiento sistemático de correspondencia entre letras y números. Esta correspondencia se presenta en la Tabla 2.2.

1	2	3	4	5	6	7	8	9
A	B	C	D	E	F	G	H	I
J	K	L	M	N/Ñ	O	P	Q	R
S	T	U	V	W	X	Y	Z	

Tabla 2.2: Correspondencia entre letras y números

A partir de esta correspondencia, se definen las cifras que servirán para determinar coordenadas del trabajo:

1. **Primera cifra:** suma del valor de las letras del nombre (*Mario*).
2. **Segunda cifra:** número de letras del primer apellido (*Gómez*).

3. **Tercera cifra:** suma del valor de las letras del nombre más el primer apellido (*Mario Gómez*).

4. **Cuarta cifra:** número de letras del nombre más el segundo apellido (*Mario Sánchez-Celemín*).

Nombre: Mario		**Primer apellido: Gómez**	
Letra	Valor	Letra	Valor
M	4	G	7
A	1	O	6
R	9	M	4
I	9	E	5
O	6	Z	8
Suma	**29**	**Suma**	**30**

Tabla 2.3: Cálculo de las cifras a partir del nombre y primer apellido

Nombre/Apellido	Longitud
MARIO	5
GÓMEZ	5
SÁNCHEZ-CELEMÍN	14

Tabla 2.4: Cálculo del número de letras

Resumen de las coordenadas

1ª Cifra	2ª Cifra	3ª Cifra	4ª Cifra
29	5	59	19

Tabla 2.5: Coordenadas calculadas a partir del nombre y apellidos

2.3. Funcionamiento del programa

El programa desarrollado para el control y simulación del robot se estructura en dos partes principales:

1. **Script LUA en CoppeliaSim:** Este script se ejecuta directamente desde CoppeliaSim al pulsar el botón de inicio de simulación. Su función principal es abrir el puerto de comunicación definido con Python y generar un archivo CSV con los datos del *heightfield* utilizado en la simulación.

2. **Script principal en Python:** Se ejecuta desde terminal mediante la estructura de módulo de Python (`-m coppeliaSim.control.main`). Este script gestiona todo el funcionamiento de la simulación y el control del robot, siguiendo los pasos resumidos a continuación:

   a) Lectura de los argumentos de terminal para seleccionar modo de ejecución, controlador y planificador.

   b) Carga del archivo de configuración en formato YAML con los parámetros de simulación y control.

   c) Ejecución del *planificador global* para generar la trayectoria del robot, dibujarla en CoppeliaSim y exportarla a `solution_path.csv`.

   d) Ejecución del *planificador local* seleccionado (PID o Pure Pursuit) para mover el robot siguiendo la trayectoria y registrar los datos de simulación.

   e) Generación de archivos de salida con los resultados de la simulación: `performance.txt`, `executed_path.csv` y `executed_twist.csv`.

En la Figura 2.2 se muestra un diagrama general del flujo de ejecución del programa.

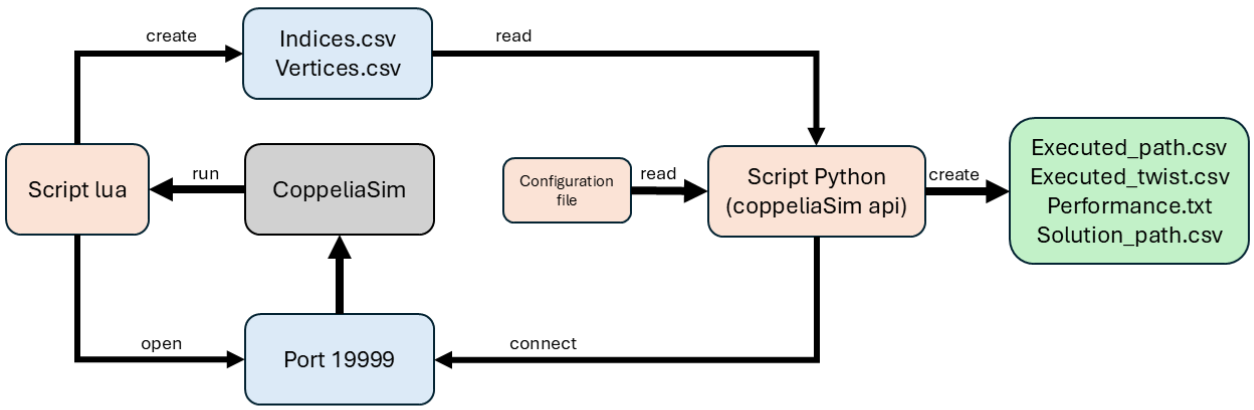


Figura 2.2: Flujo de ejecución del programa

2.3.1. Modos de configuración

La configuración del programa se realiza mediante argumentos de terminal usando la librería `argparse`, lo que permite seleccionar con facilidad los distintos modos de operación:

- **Modo de planificación:** Permite elegir entre utilizar un *planificador global* (por distancia mínima o por energía mínima) o un modo directo, donde el robot se mueve en línea recta hacia el objetivo sin generar una trayectoria global.
- **Controlador local:** Define cómo el robot sigue la trayectoria generada:
  - **Pure Pursuit:** Calcula la velocidad lineal y angular necesarias para que el robot siga un punto de referencia adelante en la trayectoria, ajustando el ángulo y la distancia a dicho punto.
  - **PID de posición:** Ajusta las velocidades lineales y angulares en función del error entre la posición y orientación actuales del robot y el punto objetivo de la trayectoria.
- **Modo directo:** El robot sigue la trayectoria de manera inmediata, sin planificador global. En este caso, el control se realiza únicamente mediante el controlador local.

Gracias a esta estructura modular, es posible combinar fácilmente distintos planificadores globales con diferentes controladores locales, facilitando pruebas comparativas y la evaluación de rendimiento en el entorno simulado.

3. Análisis y Comparación de Resultados

En este capítulo se exponen y analizan los resultados obtenidos durante la fase de simulación. Se realiza una evaluación exhaustiva del rendimiento de los controladores locales implementados (*Pure Pursuit* y *Pose PID*), así como un estudio comparativo del impacto de las distintas estrategias de planificación global (directa, mínima energía y mínima distancia) sobre el comportamiento cinemático y el coste computacional del sistema.

La Figura 3.1 muestra un ejemplo de aplicación del programa realizado sobre el simulador, donde se pueden apreciar tanto las cámaras simuladas como las rutas planificada (verde) y seguida (roja) actualizadas en tiempo real.

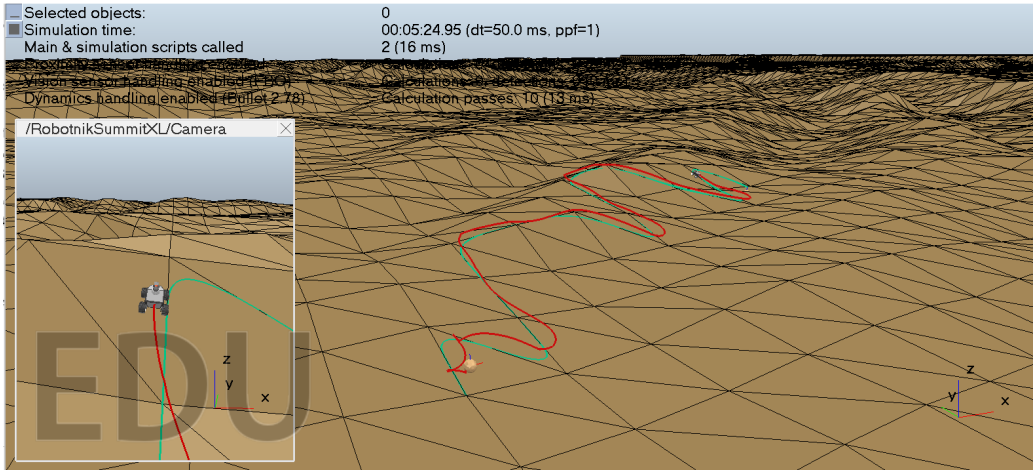


Figura 3.1: Ejemplo de aplicación sobre el simulador

3.1. Sintonización de los Controladores Locales

El paso previo a la evaluación comparativa consistió en la sintonización de los parámetros de control mediante iteraciones empíricas, buscando un equilibrio entre precisión de seguimiento y suavidad en los comandos de velocidad. Como caso de estudio, la Figura 3.2 ilustra el efecto de la variación del parámetro *lookahead distance* en el algoritmo *Pure Pursuit*.

Se evidencia que un incremento excesivo de este parámetro penaliza drásticamente el seguimiento de la trayectoria. Con un valor de $L_d = 3$, el sistema exhibe un comportamiento oscilatorio inestable al tratar las curvas de forma agresiva. Por el contrario, valores muy reducidos provocan

sobrecorrecciones. En base a este análisis, se ha establecido un valor óptimo de $L_d = 1$ para los ensayos definitivos, garantizando estabilidad sin sacrificar excesiva precisión.

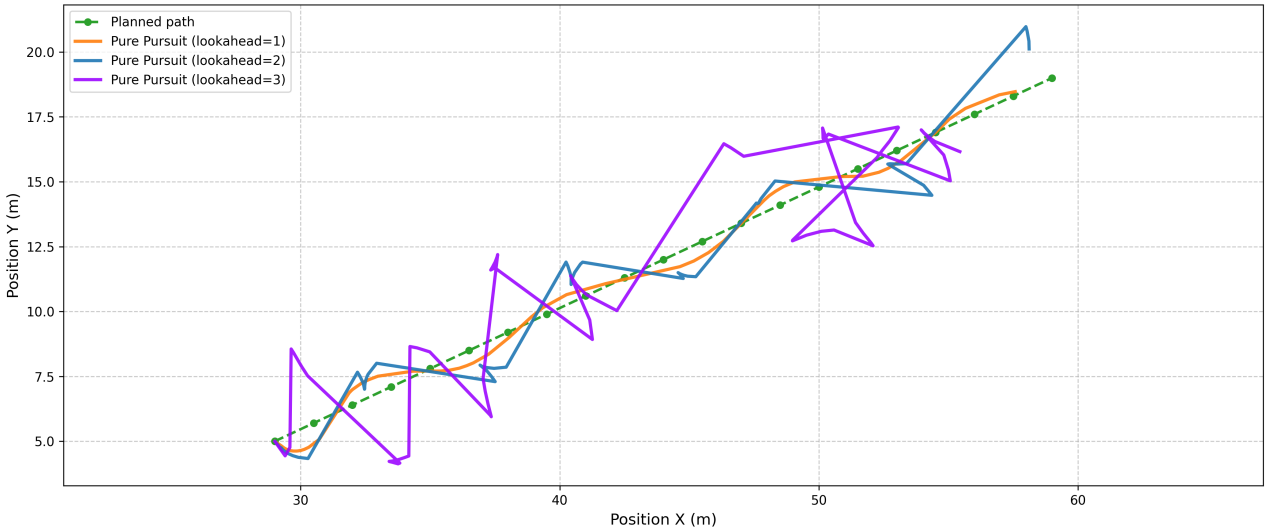


Figura 3.2: Efecto de la variación del parámetro *lookahead distance* sobre la trayectoria ejecutada por el controlador *Pure Pursuit*

Siguiendo una metodología análoga, se ajustaron las ganancias proporcionales del controlador *Pose PID* y los límites cinemáticos del robot. El conjunto final de parámetros utilizados para las simulaciones se detalla en la Tabla 3.1.

Parámetro	**Valor**
Lookahead distance (Pure Pursuit)	1.0 m
Ganancia lineal $K_{p,v}$ (Pure Pursuit)	0.5
Ganancia angular $K_{p,\omega}$ (Pure Pursuit)	0.3
Ganancia $K_{p,x}$ (Pose PID)	0.5
Ganancia $K_{p,y}$ (Pose PID)	0.5
Ganancia $K_{p,\theta}$ (Pose PID)	0.3
Velocidad lineal máxima (v_{max})	1.5 m/s
Velocidad angular máxima (ω_{max})	0.5 rad/s

Tabla 3.1: Configuración paramétrica de los controladores y restricciones cinemáticas del robot.

3.2. Evaluación en Modo Directo (Sin Planificación Global)

En este primer escenario, el robot debe alcanzar el objetivo navegando de forma reactiva entre los *waypoints* proporcionados, delegando la responsabilidad de la trayectoria exclusivamente en la lógica de los controladores locales.

Como se observa en la vista en planta de la Figura 3.3, ambos controladores logran completar el recorrido, aunque presentan ligeras oscilaciones transitorias. El *Pose PID* exhibe un comportamiento ligeramente más errático frente a giros bruscos en comparación con el *Pure Pursuit*, el cual, por su naturaleza geométrica, tiende a suavizar las transiciones entre *waypoints*.

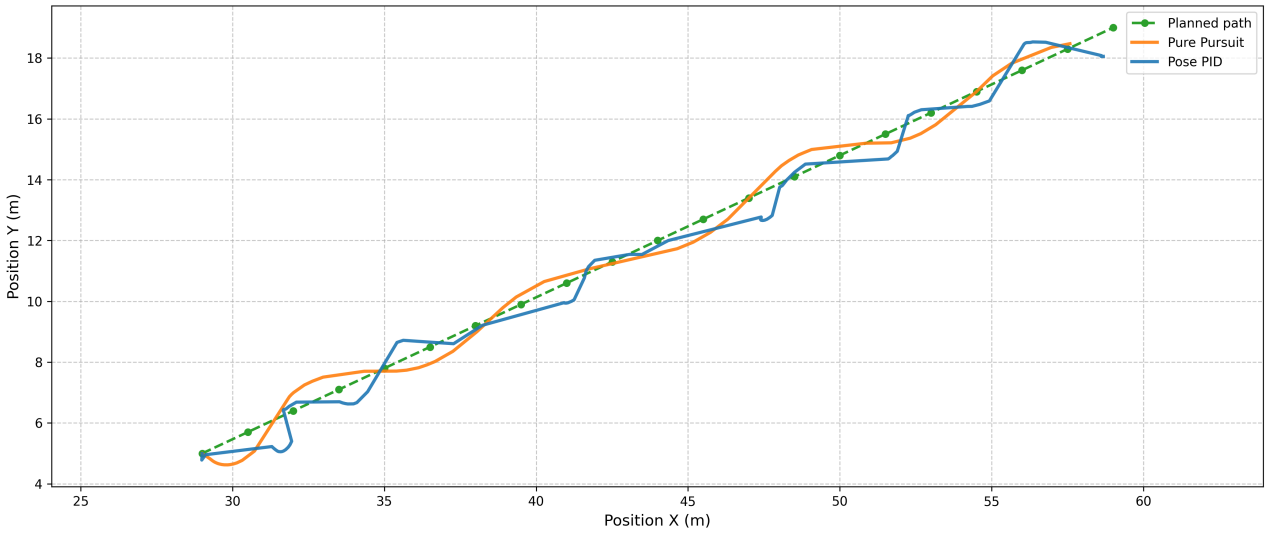
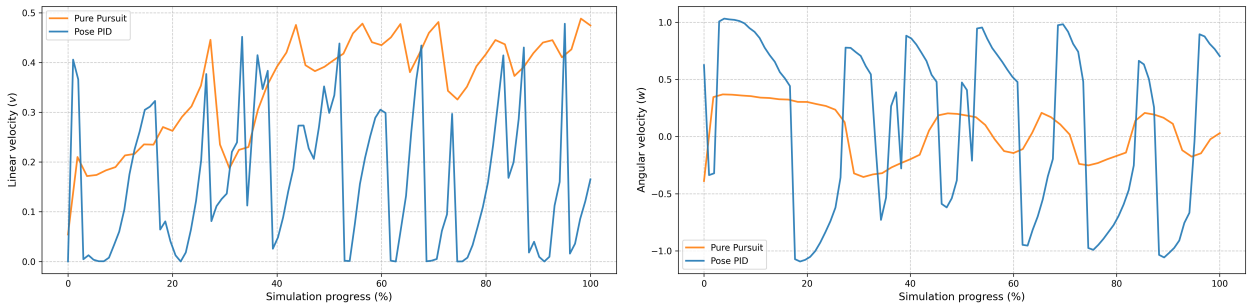


Figura 3.3: Comparativa geométrica de las trayectorias ejecutadas en modo directo.

El análisis del *twist* (velocidades de control) mostrado en la Figura 3.4 respalda esta observación geométrica. Las señales de control generadas permiten evaluar la viabilidad de implementar estos algoritmos en *hardware* real, donde se deben evitar picos de aceleración excesivos.



(a) (b)

Figura 3.4: Evolución temporal de las señales de control en modo directo: a) Velocidad lineal (v), b) Velocidad angular (ω).

En términos de rendimiento (Tabla 3.2), el *Pure Pursuit* demuestra ser notablemente más ágil en este escenario simple, completando la ruta en 45.96 s frente a los 68.87 s del *Pose PID*. Dado que no hay algoritmo de planificación global actuando, el consumo de memoria es idéntico y mínimo (58.58 MB) para ambos casos, reflejando únicamente la carga base del entorno de simulación.

Controlador Local	**Tiempo de ejecución [s]**	**Memoria [MB]**
Pure Pursuit	45.96	58.58
Pose PID	68.87	58.58

Tabla 3.2: Métricas de rendimiento computacional y temporal en modo directo.

3.3. Evaluación con Planificación Global: Criterio de Mínima Energía

Al incorporar el planificador global optimizado para minimizar el esfuerzo energético, la ruta generada impone trayectorias curvilíneas más complejas. Curiosamente, en este escenario de alta complejidad de *waypoints*, el controlador *Pose PID* demuestra una mayor fidelidad geométrica respecto a la ruta ideal (Figura 3.5), mitigando las carencias mostradas en el modo directo.

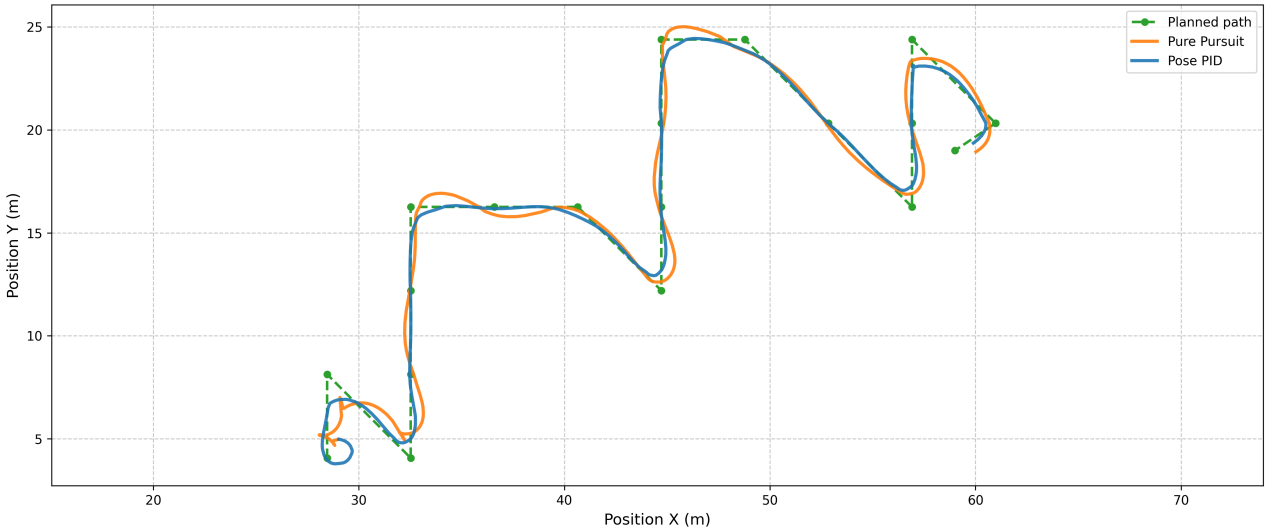
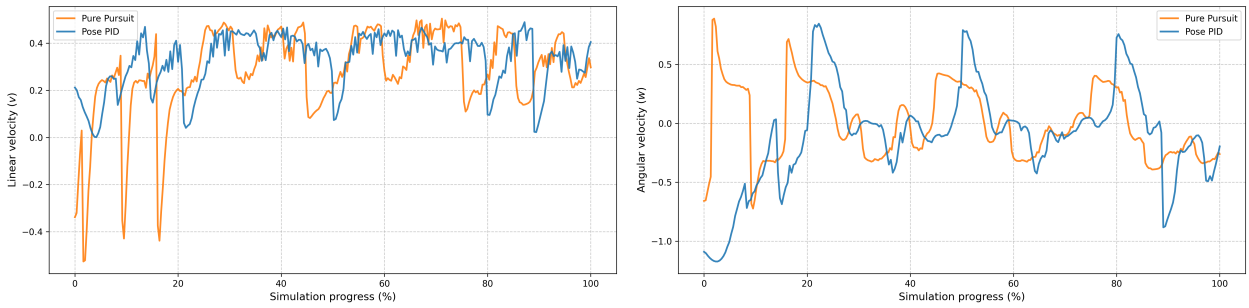


Figura 3.5: Trayectorias resultantes empleando el planificador global de mínima energía.

La Figura 3.6 muestra la respuesta de los controladores ante las exigencias de la ruta planificada.



(a) (b)

Figura 3.6: Perfil de velocidades para la ruta de mínima energía: a) Velocidad lineal (v), b) Velocidad angular (ω).

El análisis de rendimiento (Tabla 3.3) revela el alto coste asociado a esta estrategia. La búsqueda del camino energéticamente óptimo, combinada con la complejidad del seguimiento, eleva los tiempos de ejecución por encima de los 130 segundos y duplica la huella de memoria (123.4 MB) respecto al modo directo. En este caso, el *Pose PID* logra completar el circuito ligeramente más rápido (133.13 s) que el *Pure Pursuit* (138.77 s).

Controlador Local	**Tiempo de ejecución [s]**	**Memoria [MB]**
Pure Pursuit	138.77	123.4
Pose PID	133.13	123.4

Tabla 3.3: Impacto temporal y computacional empleando planificación de mínima energía.

3.4. Evaluación con Planificación Global: Criterio de Mínima Distancia

Finalmente, se evaluó el planificador global configurado para minimizar la distancia euclídea total. Como se aprecia en la Figura 3.7, las rutas generadas tienden a buscar la línea recta, resultando en trayectorias cualitativamente similares a las del modo de mínima energía, pero con giros potencialmente más pronunciados.

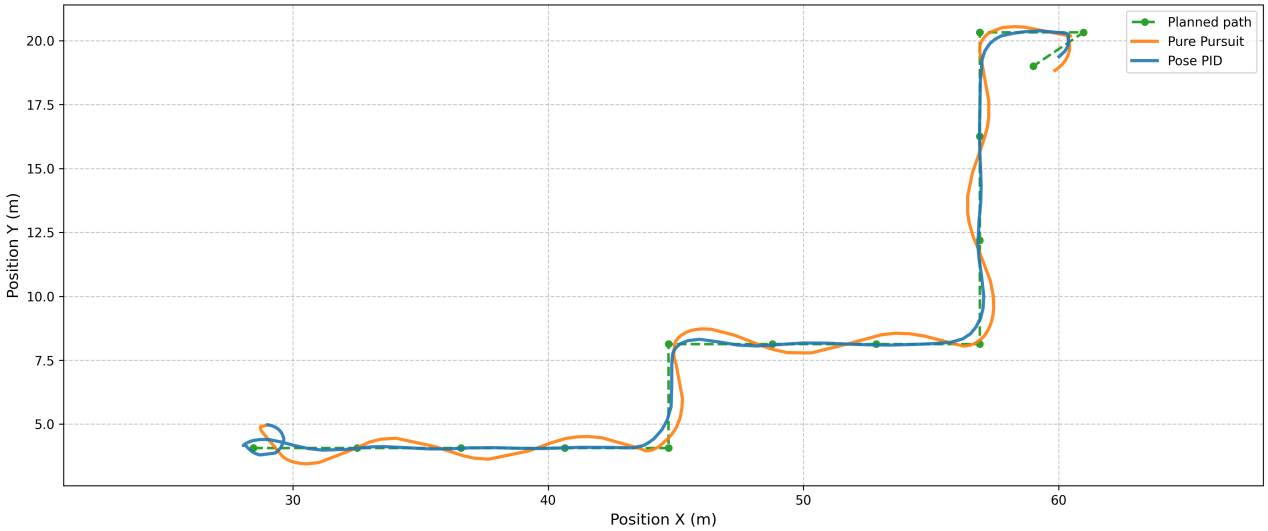
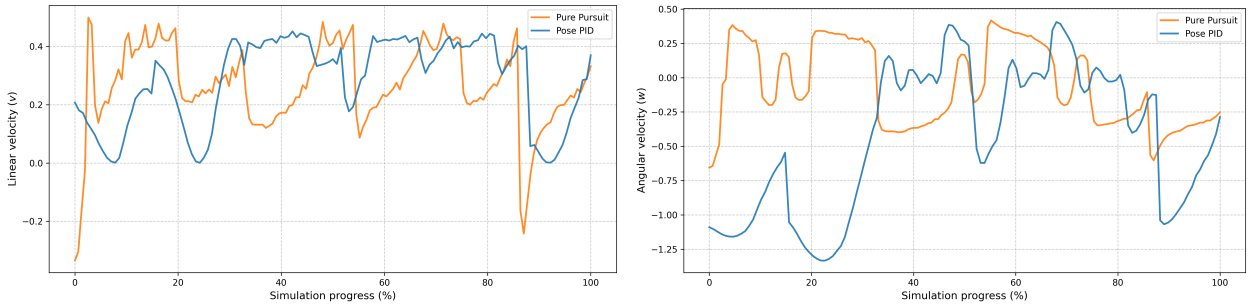


Figura 3.7: Trayectorias resultantes empleando el planificador global de mínima distancia.

La Figura 3.8 recoge las exigencias cinemáticas de este tipo de rutas sobre los controladores locales.



(a) (b)

Figura 3.8: Perfil de velocidades para la ruta de mínima distancia: a) Velocidad lineal (v), b) Velocidad angular (ω).

La principal diferencia de este algoritmo frente al de mínima energía radica en su eficiencia temporal. Según los datos recogidos en la Tabla 3.4, calcular y ejecutar el camino más corto reduce el tiempo de misión a prácticamente la mitad (aprox. 75 segundos frente a los > 130 segundos del criterio energético), manteniendo un consumo de RAM prácticamente equivalente (125.1 MB). En esta configuración, ambos controladores locales, *Pure Pursuit* y *Pose PID*, presentan un desempeño temporal virtualmente idéntico.

Controlador Local	**Tiempo de ejecución** [s]	**Memoria** [MB]
Pure Pursuit	75.28	125.1
Pose PID	74.17	125.1

Tabla 3.4: Impacto temporal y computacional empleando planificación de mínima distancia.

4. Conclusiones y Trabajos Futuros

4.1. Conclusiones

En el presente trabajo se ha llevado a cabo un diseño, implementación y evaluación exhaustiva de un sistema de navegación autónoma, contrastando el rendimiento de diferentes algoritmos de control local (*Pure Pursuit* y *Pose PID*) y estrategias de planificación global (navegación directa, mínima energía y mínima distancia). A partir del análisis de los resultados obtenidos en simulación, se extraen las siguientes conclusiones principales:

- **Desempeño de los Controladores Locales:** Ambos controladores han demostrado ser capaces de guiar al robot hacia su destino, pero presentan perfiles de comportamiento distintos. El *Pure Pursuit* destaca por su rapidez y suavidad en trayectorias de baja complejidad geométrica, minimizando el tiempo de ejecución. Por su parte, el *Pose PID* ha demostrado una fidelidad geométrica superior a la hora de seguir rutas complejas y altamente curvilíneas, aunque a costa de un mayor esfuerzo de control y ligeras oscilaciones transitorias en la velocidad angular.
- **Impacto de la Sintonización:** Se ha evidenciado que el éxito de algoritmos geométricos como el *Pure Pursuit* depende críticamente de la sintonización del parámetro *lookahead distance*. Valores excesivamente altos generan atajos inestables en las curvas (comportamiento agresivo), mientras que valores muy bajos provocan sobreoscilación. Un ajuste empírico riguroso ha sido fundamental para garantizar la estabilidad del sistema.
- **Viabilidad de los Planificadores Globales:** La introducción de una capa de planificación global impacta significativamente en el coste computacional. El planificador basado en el criterio de **mínima energía** genera rutas suaves y orgánicas, pero su elevado tiempo de cálculo (superior a 130 segundos en el entorno de pruebas) y su mayor consumo de memoria lo hacen menos viable para aplicaciones que requieran replanificación en tiempo real.
- **Estrategia Óptima:** El planificador de **mínima distancia** se erige como la solución más equilibrada. Al generar trayectorias que tienden a la línea recta, reduce a la mitad el tiempo de ejecución y cómputo respecto al algoritmo de mínima energía (aproximadamente 75 segundos), manteniendo un consumo de recursos asumible e idéntico para ambos controladores locales.

En síntesis, la elección de la arquitectura de navegación óptima dependerá de las restricciones de la misión: para entornos despejados donde prime la velocidad, una configuración de planificación directa o de mínima distancia con *Pure Pursuit* es la más eficiente; para entornos angostos o

tareas que requieran seguir trayectorias estrictas, la combinación de un planificador global con *Pose PID* garantiza un menor error de seguimiento transversal.

4.2. Líneas de Trabajo Futuro

A partir de los resultados obtenidos y las limitaciones identificadas durante el desarrollo de este proyecto, se proponen las siguientes líneas de continuación:

1. **Validación en Plataforma Física (*Sim-to-Real*):** Desplegar los algoritmos implementados en un robot móvil real para evaluar el impacto del ruido de los sensores, la odometría imperfecta y la dinámica real de los actuadores (zonas muertas, fricción) sobre los controladores sintonizados.

2. **Evasión de Obstáculos Dinámicos:** Integrar planificadores locales más avanzados, como el *Dynamic Window Approach* (DWA) o *Timed Elastic Band* (TEB), que permitan al robot reaccionar ante obstáculos móviles no mapeados en tiempo real, superando la limitación del seguimiento ciego de trayectorias globales.

3. **Análisis de Consumo Energético Real:** Realizar un estudio midiendo el par motor y el consumo eléctrico simulado/real para determinar qué controlador prolonga efectivamente la autonomía de la batería.

4. **Controladores Adaptativos:** Implementar técnicas de control donde parámetros como el *lookahead distance* del *Pure Pursuit* o las ganancias del *Pose PID* varíen dinámicamente en función de la velocidad del robot o de la curvatura del camino en ese instante específico.